



شماره دانشجویی

۸۱۰۱۰۰۲۳۰

نام درس: مبانی رایانش توزیع شده  
نام و نام خانوادگی: کسرا نوربخش



پروژه شماره ۰

## ۱ بخش اول: Implementing a key-value messaging system

### ۱.۱ مقدمه

در این بخش با استفاده از معماری توصیه شده و همچنین کد های ارایه شده برای یک پایگاه داده کلید-مقدار، به پیاده سازی آن پرداختیم.

### ۲.۱ پیاده سازی

برای بخش پایگاه داده کلید-مقدار کد ها مطالعه شد که ۴ عمل اصلی را برای این پایگاه داده به سادگی در فایل kv impl نوشته شده بود. همچنین توضیحات مربوط به این بخش هم در صورت پروژه و هم به صورت کامنت در بالای آن آمده بود که از دوباره گویی آن پرهیز شد.

#### ۱.۲.۱ استراکت keyValueServer

این استراکت اصلی سرور ما می باشد که یک نسخه از پایگاه داده را دارد. همچنین یک map از اتصالات کلاینت ها به چنل مختص به آن ها نگه می دارد. (همانطور که در صورت پروژه ذکر شده بود، برای هر کلاینت یک چنل مختص به خودش را داریم). listener در اینجا یک TCP listener برای قبول کردن اتصالات کلاینت ها می باشد. برای نگه داشتن تعداد کلاینت هایی که active و یا dropped می باشند هم دو عدد int را داریم. کلاینت ها در تست ها یک سری دستورات را می دهند که ما تمامی آن ها را با استفاده از یک بافر چنل به صورت Async هندل می کنیم. (CommandChan)

#### ۲.۲.۱ استراکت clientCommand

مشابه با استراکت توضیح داده شده بالا که مختص سرور بود، این استراکت را برای کلاینت داریم که شامل کانکشن این کلاینت، مجموعه دستورات ممکن:

□ accept: مدیریت اتصال جدید client

□ read: پردازش پیام های client

□ close: تمامی اتصالات کلاینت ها را شات داون می کند.

و در نهایت پیامی که برای هر اکشن در بالا ارسال می شود است. در ادامه ما New را داریم که کار initialization را انجام می دهد و پایگاه داده کلید-مقدار را بر می گرداند. سپس Start که بر روی پورت مشخص شده سرور را بالا می آورد و آن TCP listener که پیش تر گفته شد را initialize می کند. و runServer را برای مدیریت کار های سرور بر روی یک ترد ران می کند. Start به خوبی مشکلات احتمالی را بررسی می کند و در صورت وجود آن ها، پیام خطای مناسب می دهد. در آخر این زیر بخش هم Close را داریم که ابتدا متغیر استراکت را که برای نشان دادن اینکه یک سرور بسته است یا نه، ست می کند بعد listenre را متوقف می کند و در آخر هم یک کامند close می دهد که در بالا گفته شد.

### ۳.۲.۱ تابع runServer

حلقه‌ی اصلی سرور شامل موارد زیر است:

پذیرش اتصالات جدید: یک Goroutine برای پذیرش اتصالات جدید اجرا شده و پیام accept را به چنل ارسال می کند. پردازش دستورات در commandChan که نحوه کارکرد هر یک در بالا گفته شد.

### ۴.۲.۱ تابع handleNewClient و handleClientClose

مدیریت اتصال جدید کلاینت شامل مراحل زیر است:

ایجاد یک کانال برای ارسال پاسخ های کلاینت.

افزایش شمارنده‌ی کلاینت های فعال.

اجرای دو goroutine خواندن و نوشتن.

Goroutine خواندن:

پیام های دریافتی از کلاینت را می خواند. فضاهای اضافی را حذف کرده و read را به commandChan ارسال می کند.

اگر کلاینت قطع شود (io.EOF)، پیام close را ارسال می کند.

Goroutine نوشتن:

پاسخ ها را از کانال کلاینت دریافت و به کلاینت ارسال می کند. اگر کانال پاسخ پر باشد (شبه سازی تأخیر کلاینت

کند)، پیام های جدید را حذف می کند.

handleClientClose این تابع اتصال و کانال مربوطه را می‌بندد. شمارنده‌ی کلاینت‌های فعال و کلاینت‌های حذف شده را به روز می‌کند.

### ۵.۲.۱ تابع handleRead

این تابع پیام‌های دریافتی از کلاینت را پردازش می‌کند. پیام را با : جدا کرده و به بخش‌های مختلف تقسیم می‌کند. چهار دستوری که در ابتدای گزارش گفته شد را اجرا می‌کند. اگر پیام نامعتبر باشد، آن را در لاگ ثبت می‌کند.

### ۳.۱ بخش امتیازی

همان طور که در گروه درس اشاره شده بود، برای دستورهای پایگاه داده که از فرمت درستی برخوردار نیستند، پیام خطایی را قرار دادیم. در پیاده‌سازی تابع handleRead ابتدا بررسی می‌کنیم که دستور از حداقل ۲ بخش تشکیل شده باشد چون اگر به جز این باشد دستور غلطی است. سپس با استفاده از switch-case در هر کدام از ۴ دستور موجود، یک شرط if را داریم که اگر آرگومان‌های آن دستور مد نظر را کاربر بیشتر یا کمتر وارد کند، پیام خطایی را می‌دهیم.

### ۴.۱ ویژگی‌های کلیدی این بخش

با استفاده از goroutine و channel، چندین کلاینت به طور همزمان مدیریت می‌شوند. قبل از خاموش شدن سرور، تمام اتصالات کلاینت‌ها بسته می‌شوند. مدیریت خطاها: پیغام دادن برای خطاهای ناشی از پیام‌های نامعتبر، قطع شدن کلاینت و مشکلات I/O.

### ۵.۱ نمونه گردش کار

۱. کلاینت به سرور متصل می‌شود.
۲. کلاینت پیام Put:Key:Value را ارسال می‌کند.
۳. سرور پیام را پردازش کرده و Key:Value را در kvstore ذخیره می‌کند.
۴. کلاینت Get:Key را ارسال کرده و مقدار Key:Value را دریافت می‌کند.
۵. کلاینت قطع شده و سرور شمارنده‌ی کلاینت‌های فعال/حذف‌شده را به‌روز می‌کند.

## ۶.۱ خروجی نهایی

با استفاده از دستور `go mod init p0partA` یک ماجول ساختم و سپس با استفاده از `go test -v` به اجرای تست های داده شده پرداختیم. فقط برای اینکه بتوانیم تست ها را در سیستم عامل ویندوز اجرا کنیم، آن خطوطی از کد را که دارای سیستم کال بودند کامنت کردیم. ضمناً در نهایت با استفاده از دستور `go fmt` کد ها را فرمت کردیم. تست هایی که در این بخش بود، ۶ تست برای صحت پیاده سازی، ۲ تست برای اینکه ببینیم تعداد کلاینت هایی که هستند و رفته اند درست است و در نهایت ۲ تست کلاینت های کند را داشتیم.

```
=== RUN   TestSlowClient2
===== TestSlowClient2: 6 total clients, 4 slow client(s), 1900 Get, 100 Delete requests each =====
Server started on port 8325
All clients beginning to write...
Normal clients beginning to read...
Slow clients beginning to read...
--- PASS: TestSlowClient2 (12.55s)
Server shut down.
PASS
ok       p0partA 44.751s
```

شکل ۱: نمونه خروجی اجرا شدن تست ها و در نهایت ok کلی

## ۲ بخش دوم: تست برنامه نویسی در go

### ۱.۲ مقدمه

در این بخش از پروژه یک تست ساده برای برنامه داده شده نوشتیم.

### ۲.۲ پیاده سازی

در این بخش با الهام گیری از تست ساده داده شده یک تست ساده دیگر نوشتیم. البته به جز تستی که موجود در کد است، یک تست هم برای اطمینان از درست بسته شدن نوشته بودیم که به دلیل بدیهی بودن حذف گردید. در این تابع ما یک `go routine` را داریم که یک حلقه از منفی ۳ تا ۳ می زند و این اعداد را بر روی چنل ورودی می گذارد. سپس بررسی می شود که خروجی ها برابر با اسلایس خروجی های مد نظر است یا خیر. در این تست ما دو کیس اعداد منفی و همچنین عدد صفر را هم بررسی نمودیم. نحوه بررسی به این صورت است که در یک حلقه ساختار `case-select` را داریم که اگر چیزی در چنا خروجی بیاید آن را مقایسه می کنیم و اگر چنا خروجی خالی باشد مدت زمانی را صبر می کنیم و اگر چیزی نبود، پیام خطا چاپ می کنیم.

## ۳.۲ خروجی نهایی

با استفاده از دستور `go mod init p0partB` یک ماجول ساختیم و سپس با استفاده از `go test -v` به اجرای تست های نوشته شده پرداختیم. ضمناً در انتها با زدن دستور `go fmt` فرمت را درست کردیم.

```
PS D:\semester 8\Dist.S\Projects\P0\p0partB> go test -v
=== RUN    TestBasicCorrectness
Running TestBasicCorrectness.
--- PASS: TestBasicCorrectness (0.00s)
=== RUN    TestYourFirstGoTest
Running TestYourFirstGoTest.
--- PASS: TestYourFirstGoTest (0.00s)
PASS
ok      p0partB 0.045s
```

شکل ۲: خروجی دو تست اجرا شده

برای این پروژه، از Tutorial Go [۱] استفاده شد. همچنین از برخی از بچه ها برای دیباگ کردن و نکات پروژه من جمله نحوه اجرای این پروژه بر روی سیستم عامل ویندوز کمک گرفته شد. ضمناً تمامی نام گذاری ها CamelCase صورت گرفتند تا مشابه با استاندارد زبان Go باشد.

## مراجع

[1] "A tour of go," <https://go.dev/tour/welcome/1>.